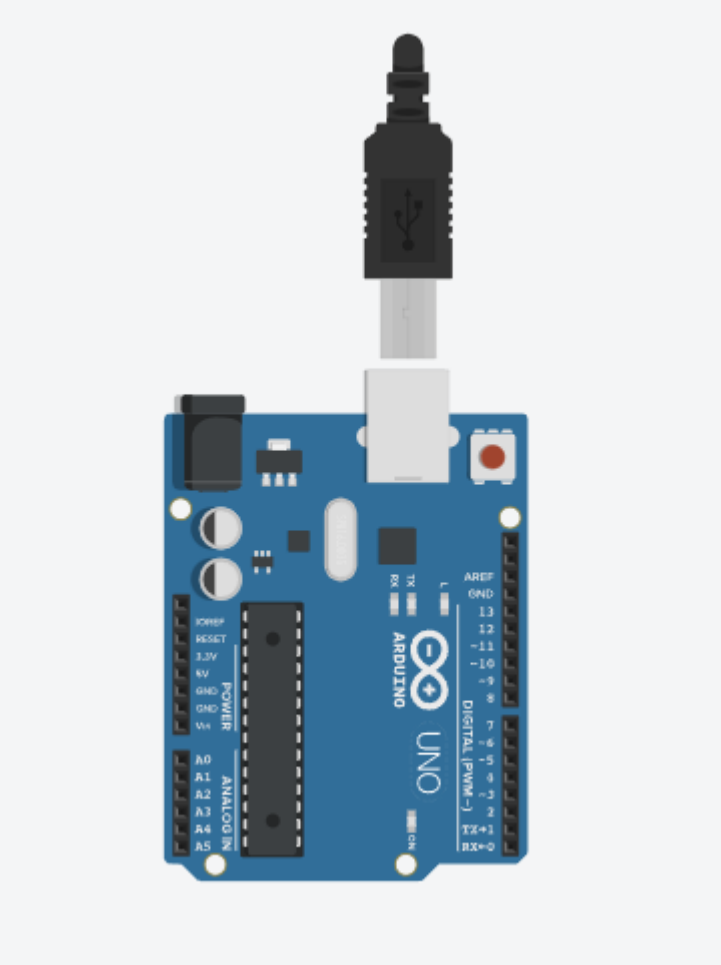

SIMPLE 3 DOF ROBOT ELECTRONICS

Student | Year 3, Semester 1

ABSTRACT

This report covers the design, building, and programming of a 3-degree of freedom (DOF) robot arm using an Arduino UNO R3. Building a robot arm is a project I have wanted to do since high school. By applying the electronics and programming skills I have learned in my mechatronics studies, I was successfully able to wire the hardware and write the control code to make the arm work. It is important to note that I am yet to complete my microcontrollers as well as the advanced manufacturing course which covers the Lagrangian mechanics of robotics.

Mcleod, Dandré
BEng Mechatronics

Contents

Table of Figure

Introduction

Ever since Grade 10, I have been curious about how robotic systems work, specifically the electronics and programming. Even though I have not yet completed my university microcontrollers course, I wanted to teach myself how sensors and microcontrollers operate. By using free online resources like YouTube and AI tools, I was able to learn the skills needed for this project on my own.

This report discusses how I developed the electronic circuit and control logic for a 3-degree of freedom (3-DOF) robotic arm. Robotic arms are essential in almost every industry today. Whether in automotive manufacturing, aerospace, or even the medical field, robotic arms are a vital piece of technology.

These systems are incredibly useful because they can take over dangerous tasks, such as lifting heavy objects safely. They are also used when a part needs to be placed with perfect accuracy, completely removing the chance of human error. Overall, robotic arms are highly efficient and can be more cost-effective than manual labour.

The following picture provides a good example of how the designed 1 Stepper - 2 Servo motor system could be implemented on a robotic arm.

Instructions on how the robotic arm will be operated:

The physical control interface utilizes two potentiometers, a single push-button, and three indicator LEDs. The push-button acts as a toggle, cycling the system through four distinct operational states:

- **State 1 (Base Control):** LED 1 is on. Potentiometer A1 controls the rotation of the stepper motor.
- **State 2 (Shoulder Control):** LEDs 1 and 2 are on. Potentiometer A0 controls servo motor 1.
- **State 3 (Wrist Control):** All three LEDs are on. Potentiometer A0 controls servo motor 2.
- **State 4 (Standby Mode):** All LEDs are off, and motor control is completely disabled.

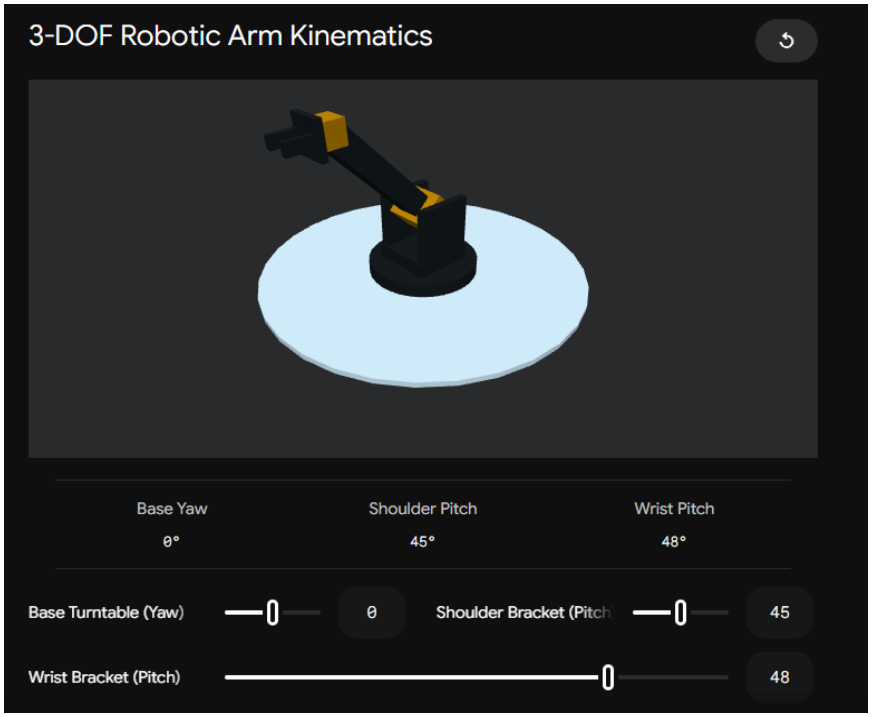

Figure 1: Simple 3 DOF Robotic Arm

Electronics and Wiring

To keep the design simple, the project required a control platform that is easy to learn and use. The Arduino UNO R3 was selected because its straightforward programming environment makes it easy to interface directly with the electronic circuits.

For this setup, the following components were selected:

- 1x Arduino UNO R3
- 2x 10K Potentiometers
- 3x 330Ω Resistors
- 3x Red LED’s
- 1x Push Button
- 2x 9-gram Micro Servo Motors (SG90)
- 1x 28BYJ-48 (5V) Stepper Motor
- 1x ULN2003 Driver Board
- 1x MB102 Power Supply Module
- Jumper Wires

The most crucial requirement is power. The UNO R3 cannot source enough current to operate all the motors simultaneously. Due to this, an external power supply was implemented using an MB102 Power Module. This module takes 6V through its DC barrel jack and steps it down to a selectable 3.3V or 5V, controlled by a manual switch. This component successfully powers the microcontroller and all external drivers, meeting the system's energy requirements.

Signal routing was divided between standard Pulse Width Modulation (PWM) for the servos and sequential digital pulsing for the stepper motor. The two SG90 servo motors, acting as the shoulder and wrist joints, are connected directly to the Arduino's digital output pins. The 28BYJ-48 stepper motor, which acts as the base turntable, interfaces with the microcontroller via the ULN2003 driver board. This driver utilizes a Darlington transistor array to amplify the low-current logic signals from the Arduino (Pins 8, 9, 10, and 11) into high-current pulses required to energize the motor's internal coils.

During the initial wiring and testing phase, a significant hardware interaction issue was observed regarding signal integrity. The analog inputs (A0 and A1) read the voltage from the 10K potentiometers using the Arduino’s internal 10-bit Analog-to-Digital Converter (ADC). Due to inherent breadboard contact resistance and electromagnetic interference from the active motors, the raw voltage signals experienced continuous high-frequency fluctuations.

While this minor electrical noise is standard, it caused severe physical instability in the stepper motor. The motor driver continuously attempted to adjust the mechanical position to match the rapidly fluctuating analog signal. This resulted in violent mechanical jittering and excessive heat generation within the motor coils due to continuous phase switching. Because this electrical noise could not be eliminated physically, a software-based deadband filter was required to stabilize the inputs.

Programming

The system's control logic was written in C++ using the Arduino Integrated Development Environment (IDE). To handle the specific pulsing requirements of the actuators, two external libraries were utilized. The standard <Servo.h> library was used to generate the absolute-positioning PWM signals for the shoulder and wrist joints. For the base turntable, the <AccelStepper.h> library was implemented. This library was chosen over standard stepper libraries because it supports non-blocking mathematical execution, allowing for smooth acceleration profiles without halting the rest of the program.

To control three independent motors using only two potentiometers and one push-button, the code was structured as a Finite State Machine (FSM). Rather than running a single continuous loop of commands, the system is divided into four isolated operational states:

State 1: Stepper Motor active (Potentiometer A1).

State 2: Servo 1 active (Potentiometer A0).

State 3: Servo 2 active (Potentiometer A0).

State 0: System Standby (Motors hold position, inputs ignored).

A global integer variable tracks the current state. When the FSM enters a specific state, it executes the movement commands for that specific motor and illuminates the corresponding LED combination on the control panel, effectively multiplexing the analog inputs.

A critical software requirement was eliminating the use of the standard delay() function. Because the <AccelStepper.h> library calculates motor steps in the background, pausing the microcontroller for even a few milliseconds causes the motor to stutter and freeze.

To achieve a non-blocking architecture, the push-button input relies on a State Change Detection algorithm. Instead of checking if the button is currently being held down, the logic compares the current button state to its previous state. The FSM only increments to the next state when it registers the exact millisecond the button transitions from HIGH (unpressed) to LOW (pressed). This allows the main loop to run continuously at maximum speed, ensuring the stepper motor pulses remain perfectly timed and smooth.

Conclusion

Building this 3-axis robotic arm proved that the Arduino UNO R3 is a great, easy-to-use control platform. By using a state machine and non-blocking code, the system successfully controls both stepper and servo motors without making the hardware setup too complicated. This prototype is a successful example of linking basic electronics with practical control systems.

Even though the build was successful, testing showed two main issues that need fixing. First, the control system does not know the physical limits of the arm. This means the user can turn the arm too far, which could place too much stress on the joints and break the brackets.

Second, using one potentiometer (A0) to control multiple servos causes a problem when switching states. When you switch to a new servo, it instantly snaps to the potentiometer's current position instead of starting smoothly from where it was last left.

To make the arm safer and more efficient, future versions will include software limits and physical limit switches to physically stop the arm from moving too far. To fix the snapping issue, the control panel needs a separate potentiometer for each motor. This will ensure the Arduino remembers the exact position of every joint.

Overall, this prototype successfully demonstrated how to integrate multiple motors and filter out electrical noise. The next step is to upgrade the code so the arm can automatically track XYZ coordinates, rather than relying on manual joint controls.

Figure 2: Stepper- and Servo motors

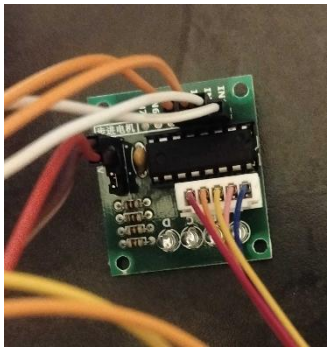

Figure 3: ULN2003 Driver for Stepper motor

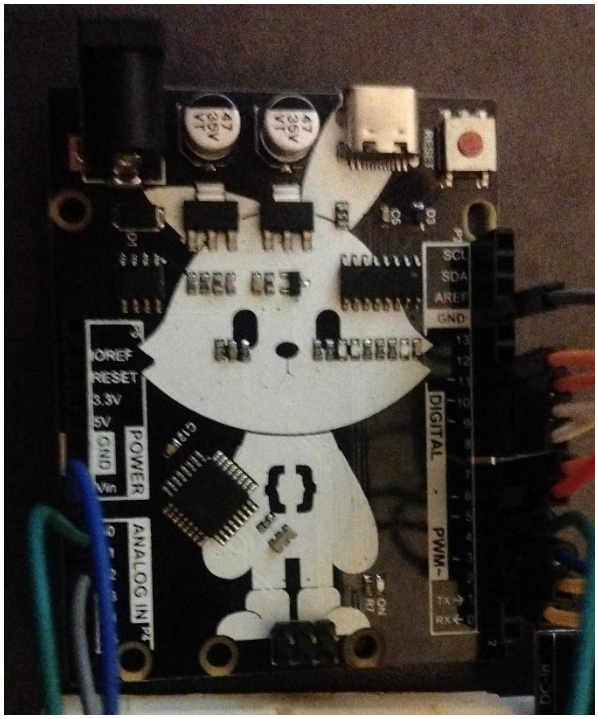

Figure 4: Arduino UNO R3 Clone

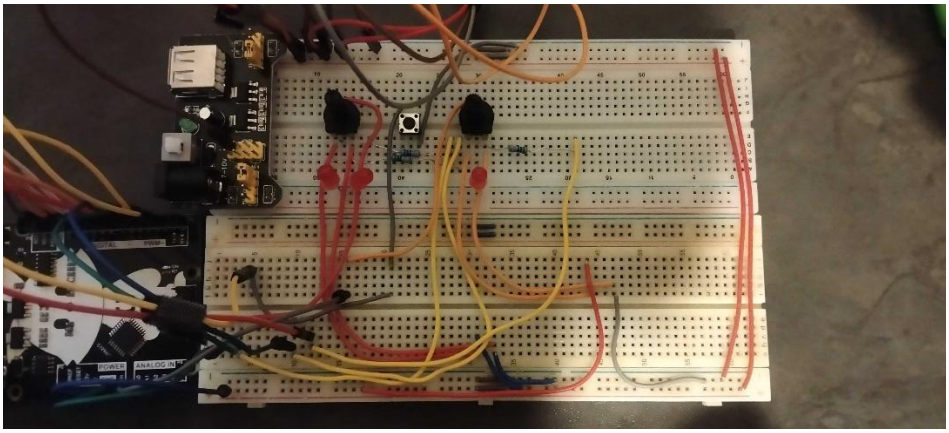

Figure 5: Breadboard and Wiring

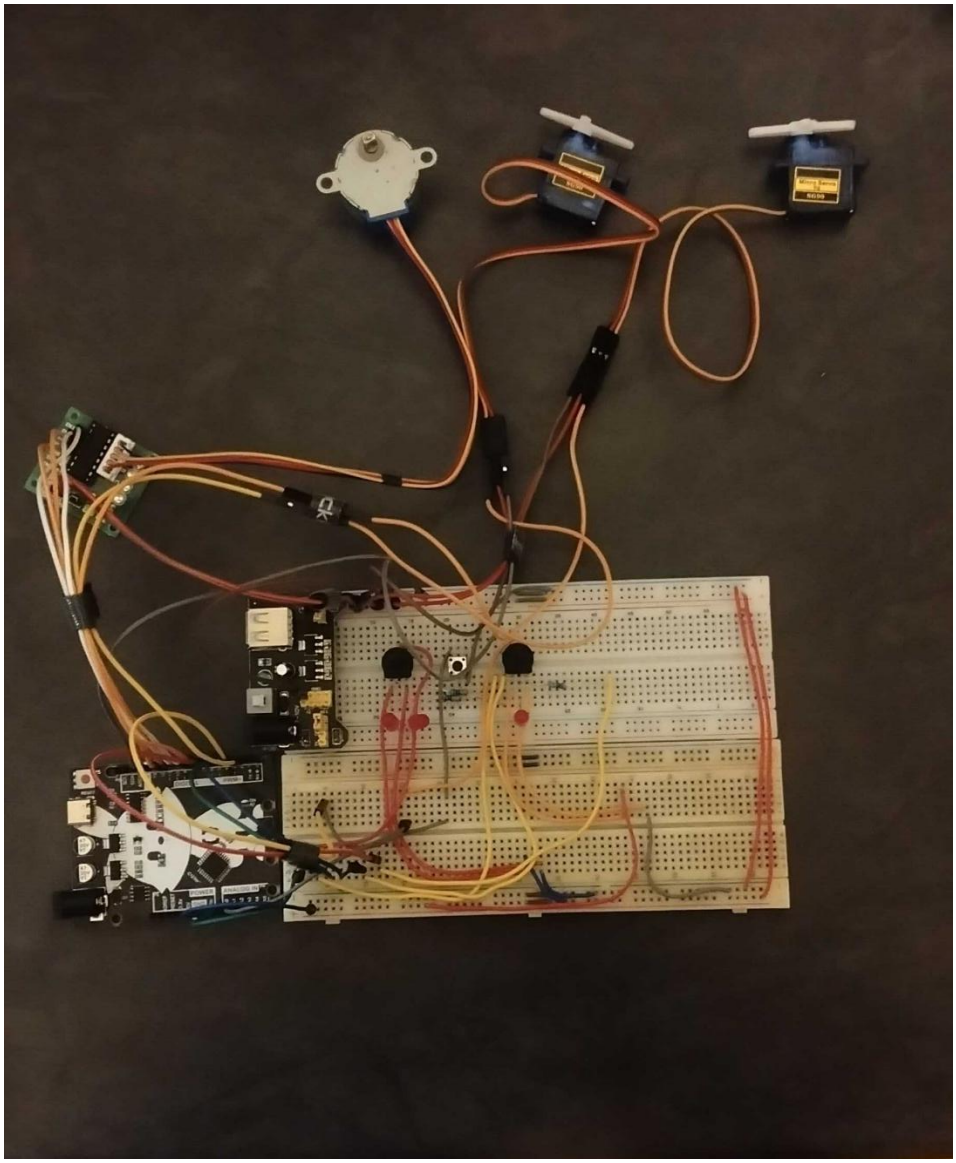

Figure 6: Whole Electronic Circuit

Arduino C++ Code

```
#include <Servo.h>
#include <AccelStepper.h>

Servo servo1;
Servo servo2;
AccelStepper myStepper(AccelStepper::FULL4WIRE, 8, 10, 9, 11);

int buttonState = 1;
int lastButtonRead = HIGH;
int lastPot2Val = 0;
const int noiseThreshold = 10;

const int buttonPin = 3;
const int led1 = 4;
const int led2 = 5;
const int led3 = 6;
const int pot1 = A0;
const int pot2 = A1;
const int servo1Pin = 7;
const int servo2Pin = 2;

void setup() {
  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  servo1.attach(servo1Pin);
  servo2.attach(servo2Pin);

  myStepper.setMaxSpeed(1000.0);
  myStepper.setAcceleration(400.0);
}

void loop() {
  int currentButtonRead = digitalRead(buttonPin);

  if (lastButtonRead == HIGH && currentButtonRead == LOW) {
    buttonState++;
  }
  lastButtonRead = currentButtonRead;

  if (buttonState == 1) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, LOW);
```

```
    digitalWrite(led3, LOW);

    int pot2Val = analogRead(pot2);

    if (abs(pot2Val - lastPot2Val) > noiseThreshold) {
      int stepperTarget = map(pot2Val, 0, 1023, 0, 2048);

      myStepper.enableOutputs();
      myStepper.moveTo(stepperTarget);

      lastPot2Val = pot2Val;
    }
  }

  else if (buttonState == 2) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, LOW);

    int pot1Val = analogRead(pot1);
    int angle1 = map(pot1Val, 0, 1023, 0, 180);
    servo1.write(angle1);
  }

  else if (buttonState == 3) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, HIGH);

    int pot1Val = analogRead(pot1);
    int angle2 = map(pot1Val, 0, 1023, 0, 180);
    servo2.write(angle2);
  }

  else {
    buttonState = 0;
    digitalWrite(led1, LOW);
    digitalWrite(led2, LOW);
    digitalWrite(led3, LOW);
  }

  if (myStepper.distanceToGo() == 0) {
    myStepper.disableOutputs();
  }

  myStepper.run();
}
```

Electrical Drawing